

Comentarios en C++

Desde principiante hasta nivel práctico

Enfoque: Desarrollo de C++

Capítulo técnico, educativo y práctico para comprender, aplicar y mantener comentarios profesionales en programas C++.

Objetivo del capítulo

Aprenderás qué son los comentarios en C++, por qué existen, cómo se escriben y cómo utilizarlos profesionalmente para explicar decisiones, reglas, limitaciones y partes importantes de un programa. Los comentarios no son instrucciones ejecutables: son información dirigida a las personas que leen y mantienen el código.

Al finalizar este capítulo serás capaz de:

- Definir un comentario y diferenciarlo del código ejecutable.
- Utilizar `//` para comentarios de una línea.
- Utilizar `/* ... */` para comentarios de varias líneas.
- Explicar qué ocurre durante el preprocesamiento.
- Documentar variables, decisiones, fórmulas, unidades y limitaciones.
- Detectar comentarios redundantes, incorrectos, desactualizados o inseguros.
- Aplicar comentarios en programas de máquinas, sensores, inventarios y producción.
- Construir y revisar un mini proyecto documentado.

Introducción al tema

¿Qué es un comentario?

Un comentario es un texto incluido en el archivo fuente para explicar el programa. El compilador lo ignora como instrucción. Una analogía: en el plano de una casa, las paredes representan el código y las notas del arquitecto representan los comentarios. Las notas no construyen la casa, pero explican decisiones importantes.

¿Por qué existe?

Los programas crecen y pueden ser modificados por personas distintas a su autor. Un comentario conserva el contexto de una decisión, una regla de negocio, una unidad de medida, una limitación o una advertencia.

¿Qué problema resuelve?

- Reduce el tiempo para entender código existente.
- Facilita el trabajo en equipo.
- Explica decisiones que no son evidentes.
- Ayuda durante revisiones y mantenimiento.
- Señala limitaciones o tareas pendientes.

¿Dónde se utiliza?

En aplicaciones empresariales, automatización industrial, PLC, sensores, robots, drones, videojuegos, inventarios, sistemas de producción y software de propósito general.

Relación con el curso

Los comentarios acompañan la sintaxis, las salidas con `cout`, las variables, las entradas con `cin`, los tipos de datos, las condiciones, los bucles, las funciones y posteriormente las clases. No sustituyen esos conceptos; los explican.

Explicación completa y progresiva

1. Comentario de una línea

Qué es: comienza con `//` y termina al final de la línea. **Por qué se utiliza:** permite explicar una instrucción breve. **Cuándo:** junto a variables, operaciones o decisiones. **Cómo funciona:** el preprocesador ignora el resto de la línea.

```
// Comentario independiente
int edad = 35; // Explica el dato almacenado
```

Ejemplo básico

```
#include <iostream>
using namespace std;

int main() {
    // Guardamos la edad del usuario
    int edad = 35;

    cout << edad << endl;
    return 0;
}
```

Línea por línea: la biblioteca permite usar `cout`; `main` es el inicio; el comentario explica la variable; `int edad` crea un entero; `cout` muestra 35; `return 0` termina correctamente. El comentario no aparece en pantalla.

2. Comentario de varias líneas

Qué es: un bloque entre `/*` y `*/`. **Cuándo:** encabezados, explicaciones extensas o bloques temporales. **Riesgo:** olvidar el cierre puede ocultar código y producir errores.

```
/*
    Sistema de monitoreo
    Datos simulados para aprendizaje
*/
```

3. Comentarios al final de una instrucción

```
double temperatura = 72.5; // Grados Celsius
int velocidad = 1500;       // Revoluciones por minuto
```

Son útiles cuando la unidad o el propósito no es evidente. No deben ser tan largos que dificulten leer el código.

4. Comentarios de intención

El código muestra qué hace una instrucción; un comentario útil explica por qué se tomó una decisión.

```
// Se deja un margen del 10% para evitar operar cerca del límite.
double limiteSeguro = temperaturaMaxima * 0.90;
```

5. Comentarios de advertencia

```
// La lectura requiere que el sensor esté conectado.
double temperatura = leerTemperaturaSensor();
```

La advertencia ayuda al desarrollador, pero no reemplaza una validación ni un mecanismo de seguridad.

¿Qué ocurre internamente?

Antes de compilar, C++ pasa por el preprocesamiento. En esa etapa los comentarios se eliminan del texto que continuará hacia el compilador.

```
Fuente:
int piezas = 100; // Cantidad inicial
```

```
Texto procesado:
int piezas = 100;
```

Por eso los comentarios no producen salida ni instrucciones de ejecución. Sin embargo, un comentario incorrecto puede provocar errores humanos durante el mantenimiento.

Ejemplos progresivos

Nivel 1 — Básico

```
#include <iostream>
using namespace std;

int main() {
    // Mostramos un saludo
    cout << "Hola, C++" << endl;
    return 0;
}
```

Resultado: Hola, C++. Error común: confundir // con cout. El primero se ignora; el segundo muestra texto.

Nivel 2 — Variables y entradas

```
#include <iostream>
using namespace std;

int main() {
    // Nombre que se recibirá del operador
    string nombre;
    cout << "Nombre: ";
    cin >> nombre;
    // Mostramos el dato recibido
    cout << "Registrado: " << nombre << endl;
    return 0;
}
```

Los comentarios explican declarar, solicitar y mostrar. Deben coincidir con lo que realmente hace el programa.

Nivel 3 — Máquina industrial

```
#include <iostream>
using namespace std;

int main() {
    // Datos simulados de una máquina
    string maquina = "Torno CNC";
    double temperatura = 82.5;
    int piezas = 450;
    bool activa = true;

    // La máquina activa puede continuar produciendo
    if (activa) {
        cout << "Máquina en operación" << endl;
    } else {
        cout << "Máquina detenida" << endl;
    }

    // Resumen para el operador
    cout << maquina << " - " << temperatura << " C - "
        << piezas << " piezas" << endl;
    return 0;
}
```

La estructura separa datos, decisión y resumen. Esto facilita localizar una sección cuando el proyecto crece.

Nivel 4 — Estilo profesional

```
/*
    Sistema: Monitoreo de producción
    Propósito: mostrar el estado de una máquina.
    Nota: los datos son simulados.
*/
#include <iostream>
#include <string>
using namespace std;

int main() {
    const double LIMITE_TEMPERATURA = 80.0;
```

```

string maquina = "Torno CNC";
double temperatura = 82.5;

// La alerta se activa al superar el límite definido.
bool alerta = temperatura > LIMITE_TEMPERATURA;

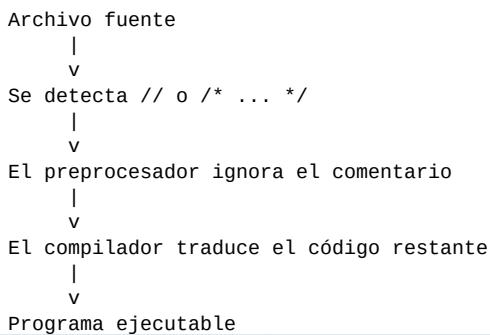
cout << "Máquina: " << maquina << endl;
if (alerta)
    cout << "ALERTA: revisar temperatura" << endl;
else
    cout << "Temperatura normal" << endl;

return 0;
}

```

El encabezado explica propósito y limitación. El comentario importante documenta la regla de negocio. En un sistema real, la alarma y la seguridad deben implementarse mediante código y hardware apropiado.

Diagramas y comparaciones



Elemento	¿Se ejecuta?	Propósito
int piezas = 100;	Sí	Guardar un dato.
cout << piezas;	Sí	Mostrar información.
// Cantidad inicial	No	Explicar el dato.
/* Regla */	No	Documentar una sección.

Concepto	Cuándo usarlo	Diferencia
//	Nota breve	Termina al final de la línea.
/* ... */	Bloque o encabezado	Necesita cierre explícito.
cout	Salida al usuario	Es código ejecutable.
Nombre descriptivo	Siempre que sea posible	Comunica sin comentario adicional.

Errores de confusión comunes

- Un comentario no es una salida: // Hola no aparece en pantalla.
- El texto entre comillas no es comentario: cout << "// texto"; muestra el texto.
- Comentar código no equivale a validarlo.
- Un nombre descriptivo no debe sustituirse por un comentario enorme.
- Una advertencia escrita no garantiza seguridad.

Errores comunes al trabajar con Comentarios en C++

- **Olvidar */:** el comentario puede extenderse y ocultar código. Revisa cada apertura y cierre.
- **Contradecir el programa:** una unidad o regla incorrecta puede causar decisiones equivocadas. Actualiza el texto.
- **Comentar cada línea:** genera ruido y no aporta comprensión. Comenta decisiones, no símbolos evidentes.
- **Ocultar código roto:** usar comentarios como almacenamiento permanente dificulta el mantenimiento. Usa control de versiones.
- **Incluir secretos:** nunca guardes contraseñas, tokens o claves en comentarios.
- **Escribir párrafos interminables:** mejora primero nombres, funciones y estructura.

Buenas prácticas

- Explica intención y motivo, no solamente la operación.
- Usa nombres claros como `temperaturaMaximaPermitida`.
- Mantén el comentario cerca del código relacionado.
- Actualiza comentarios cuando cambia la lógica.
- Documenta reglas de negocio, unidades, supuestos y limitaciones.
- No repitas literalmente lo que ya dice el código.
- Usa un estilo consistente en todo el proyecto.
- Utiliza TODO con moderación y revísalo periódicamente.
- No incluyas información sensible.
- Recuerda que los comentarios no reemplazan validaciones ni mecanismos de seguridad.
- En proyectos grandes puedes utilizar documentación automática como Doxygen.

Conectando conocimientos

Tema anterior	Relación
Sintaxis	Los delimitadores deben estar bien escritos.
Salidas	Explican qué información se muestra.
Variables	Documentan significado y unidades.
Entradas	Explican qué dato se solicita y qué se espera.
Tipos de datos	Aclaran si un valor es texto, cantidad, medida o estado.
if/else	Explican reglas de decisión.
Bucles	Aclaran el proceso que se repite.

Ejercicios prácticos

Realiza los ejercicios en orden. No se incluyen soluciones inmediatas.

Ejercicio 1 — Comprensión básica

Crea un programa que muestre nombre, edad y ciudad. Agrega tres comentarios de una línea. Objetivo: distinguir código y comentarios. Pista: explica intención, no cada símbolo.

Ejercicio 2 — Aplicación

Registra nombre de máquina, temperatura y piezas. Incluye un encabezado `/* ... */` y comentarios de unidades. No repitas literalmente los nombres.

Ejercicio 3 — Integración

Solicita una temperatura y determina si está dentro de un rango usando `cin` e `if/else`. Comenta la regla, pero no reemplaces la validación.

Ejercicio 4 — Caso práctico

Documenta una estación que registra piezas producidas, rechazadas y estado. Explica la diferencia entre producidas y aceptadas.

Ejercicio 5 — Reto

Revisa un inventario con comentarios incorrectos, repetitivos y desactualizados. Corrige al menos cinco, elimina los innecesarios y documenta dos reglas sin cambiar el comportamiento.

Mini proyecto del capítulo

Nombre: Registro documentado de una estación de producción

Contexto: una empresa necesita una aplicación de consola que simule el registro de una máquina. **Objetivo:** aplicar variables, entradas, salidas, decisiones y comentarios profesionales.

Requisitos funcionales

- Solicitar nombre, modelo, temperatura, piezas y estado.
- Mostrar un resumen ordenado.
- Emitir alerta si la temperatura supera un límite.
- Indicar si la estación está activa o detenida.

Requisitos técnicos

- Usar `iostream` y tipos adecuados.
- Incluir comentarios de una línea y uno de varias líneas.
- Documentar unidades y reglas.
- No incluir datos sensibles.
- No comentar cada línea.

Etapas

- Paso 1: crea el encabezado y la estructura de `main`.
- Paso 2: declara variables y solicita datos.
- Paso 3: agrega el límite y la regla de alerta.
- Paso 4: muestra el resumen.

- Paso final: revisa que cada comentario sea correcto, útil y actualizado.

Pistas: usa nombres como `temperaturaActual` y `piezasProducidas`; comenta por qué existe una alerta; si una explicación es demasiado larga, mejora primero la estructura del programa.

Comprueba tus conocimientos

1. ¿Qué es un comentario?
2. ¿Qué símbolos se usan para una línea?
3. ¿Qué símbolos delimitan varias líneas?
4. ¿Se ejecutan los comentarios?
5. ¿Qué ocurre si falta `*/`?
6. ¿Cuál es la diferencia entre `//` y `cout`?
7. ¿Por qué evitar comentarios redundantes?
8. ¿Qué información merece documentarse?
9. ¿Por qué un comentario incorrecto es peligroso?
10. ¿Es correcto guardar una contraseña en un comentario?
11. ¿Qué problema hay en `int piezas=100; // muestra temperatura`?
12. ¿`cout << "// texto";` es comentario?
13. ¿Cómo ayudan al mantenimiento?
14. ¿Qué hacer cuando cambia la lógica?
15. ¿Por qué un comentario no reemplaza una validación?

Respuestas y explicaciones

1. Texto no ejecutable que explica el programa.
2. `//`.
3. `/*` y `*/`.
4. No; se eliminan durante el preprocesamiento.
5. Puede ocultar código y causar errores.
6. `//` se ignora; `cout` muestra información.
7. Porque repiten algo evidente y generan ruido.
8. Intención, reglas, unidades, supuestos, limitaciones y advertencias.
9. Puede inducir a modificar mal el sistema.
10. No; puede exponer secretos.
11. El comentario no corresponde con la operación.
12. No; dentro de comillas es una cadena.
13. Conservan contexto y reducen tiempo de comprensión.
14. Actualizarlo o eliminarlo.
15. Porque la seguridad debe implementarse con código y mecanismos apropiados.

Resumen del capítulo

Concepto	Sirve para	Error común
<code>//</code>	Notas breves	Creer que continúa en otra línea.
<code>/* ... */</code>	Bloques y encabezados	No cerrar el bloque.
Intención	Explicar decisiones	Repetir el código.
Advertencia	Documentar limitaciones	Sustituir validaciones.
Nombre descriptivo	Comunicar propósito	Usar nombres ambiguos.

Ideas clave que debes recordar

- Los comentarios están dirigidos a personas.
- `//` termina al final de la línea.
- `/* ... */` necesita cierre.
- Comenta intención y reglas.
- Mantén los comentarios actualizados.
- No guardes secretos.
- Un buen nombre reduce comentarios innecesarios.
- Los comentarios no sustituyen seguridad ni validación.
- La calidad importa más que la cantidad.

Mapa de aprendizaje

Conocimientos previos
Sintaxis + Salidas + Variables + Entradas
|
v
Comentarios de una y varias líneas
|
v
Explicar variables e instrucciones
|
v
Documentar reglas y decisiones
|
v
Integrar en producción, inventarios y sistemas
|
v
Código profesional y mantenible
|
v
Siguiendo temas: operadores, decisiones,
bucles, funciones y P00

El tema prepara al estudiante para escribir programas más grandes y comprensibles. A partir de ahora, cada nuevo concepto podrá documentarse con comentarios útiles y profesionales.